Point Operations & Histograms

Week 2 — Session 2: Thresholding & advanced enhancement

Practical Image Processing

Lecture 2.2 ▪ 60–70 min

Recap of Session 1

- The **histogram** diagnoses exposure/contrast *before* you touch a pixel.
- **Point operations** $s = T(r)$: brightness, contrast, negative, gamma — all a LUT.
- **Stretch** and **equalization** spread a bunched histogram across 0–255.
- Today: turn those enhanced grays into a clean **binary mask**, then go deeper (CLAHE, matching, color).

The goal

DocScan needs the meter digits as a crisp black-on-white mask for the next stages.

Roadmap

Thresholding

Adaptive thresholding

Advanced enhancement

Color

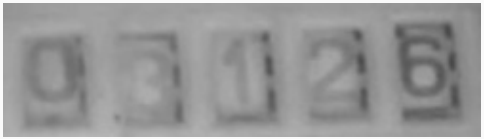
In-class exercise

Putting it together

Wrap-up

Thresholding

Thresholding: intensity → two classes



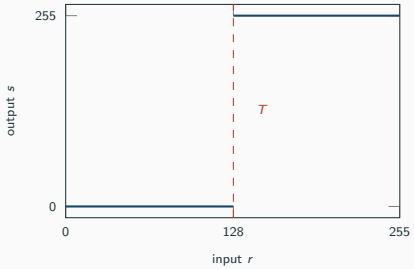
grayscale meter



Otsu binary: digits vs background

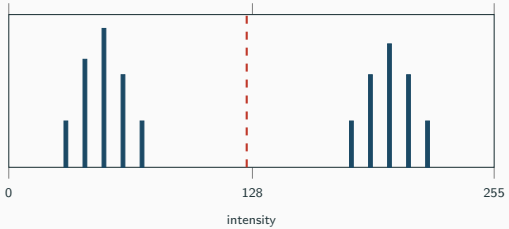
- $s = 255$ if $r > T$ else 0. Turns gray into black/white.
- The whole game is choosing T. A bimodal histogram makes it easy.
- This is DocScan's "make it crisp" step — digits become a clean mask.

Threshold as a transfer function



- A **step** at T: the most extreme point operation.
- Everything below T collapses to one class, everything above to the other.
- All the subtlety moves into *where* to put the step.

Choosing T from a bimodal histogram



- Two humps = object vs background. The **valley** between them is the natural T.
- Put T in the valley $\rightarrow$ few pixels are misclassified.
- But eyeballing the valley doesn't scale — we want it *automatic*.

Otsu: pick the threshold automatically

- Otsu's method finds the T that best **separates** the two histogram humps.
- Formally: minimize the within-class variance (equivalently, maximize between-class variance).
- One line: `cv2.threshold(g, 0, 255, THRESH_BINARY+THRESH_OTSU)`.

Math

Between-class variance at threshold T: $\sigma_b^2(T) = \omega_0(T)\,\omega_1(T)\,\big[\mu_0(T) - \mu_1(T)\big]^2$, where ω_i are class weights and μ_i class means. Otsu picks $T^\star = \arg\max_T \sigma_b^2(T)$.

Why maximize between-class variance?

- Total variance $\sigma^2 = \sigma_w^2(T) + \sigma_b^2(T)$ is *constant* (doesn't depend on T).
- So minimizing **within**-class spread $\equiv$ maximizing **between**-class separation.
- Between-class is cheap: just class weights ω_i and means μ_i, both from the histogram.
- Sweep $T = 0 \ldots 255$, evaluate $\sigma_b^2(T)$, keep the best — a 256-step loop.

✓ Good practice

Otsu needs *no parameters* and runs on the histogram alone — perfect for a bimodal meter crop.

Worked example: Otsu on two classes

Math

3 dark pixels at $\{40, 50, 60\}$ and 3 bright at $\{200, 210, 220\}$; try $T = 125$.

- Weights: $\omega_0 = \frac{3}{6} = 0.5$, $\omega_1 = 0.5$.
- Means: $\mu_0 = \frac{40+50+60}{3} = 50$, $\mu_1 = \frac{200+210+220}{3} = 210$.
- $\sigma_b^2 = 0.5 \cdot 0.5 \cdot (50 - 210)^2 = 0.25 \cdot 25600 = 6400$ — large & maximal, since any other T moves a pixel to the "wrong" class and shrinks $(\mu_0 - \mu_1)^2$.

Key idea

The best split sits in the empty valley — exactly where the two means are farthest apart.

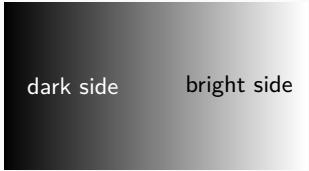
Otsu in code (and reading the result)

```python
T, binary = cv2.threshold(gray, 0, 255,
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print('Otsu chose T =', T)          # the automatic threshold
# invert if digits came out white-on-black:
# binary = cv2.bitwise_not(binary)
```

Note

Pass `0` as the threshold arg — the `OTSU` flag tells OpenCV to compute T for you and return it.

When a single global T fails



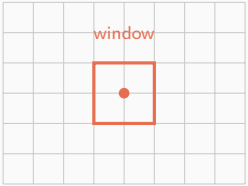
a shadow gradient across the page

✗ Pitfall

Otsu assumes two clear classes with even lighting. Under a shadow gradient, one global T blackens the dark side and whitens the bright side — digits vanish on both.

Adaptive thresholding

Adaptive thresholding: a local T per neighbourhood



T = local mean $- C$, computed per pixel

- Slide a window; threshold each pixel against **its own** neighbourhood.
- Local mean (or Gaussian-weighted mean) tracks the lighting gradient.
- Subtract a constant C to bias against noise in flat regions.

Adaptive thresholding for uneven light

```
# a local threshold per neighbourhood -> robust to shadows/gradients
binary = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, blockSize=31, C=10)
```

Key idea

Global Otsu for even lighting; adaptive for real photos with shadows. DocScan photos usually need adaptive.

Tuning blockSize and C

- `blockSize` = neighbourhood size (odd). Too small → speckle; too large → acts global again.
- `C` = margin subtracted from the local mean. Larger C → fewer white pixels (kills noise, may thin digits).
- Rule of thumb: `blockSize` a bit larger than the stroke width you want to keep.

✗ Pitfall

`blockSize` too small turns texture/noise into a confetti of speckles.

✓ Good practice

Enhance first (gamma/CLAHE), *then* threshold — a cleaner input makes tuning far easier.

Advanced enhancement

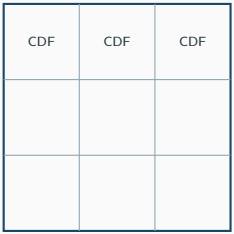
CLAHE: local, limited equalization

- Global equalization over-amplifies noise in flat regions.
- **CLAHE** equalizes in small tiles and **clips** the histogram to limit amplification.
- Usually the better default for real photos: `cv2.createCLAHE(clipLimit=2.0)`.

✓ Good practice

For DocScan, CLAHE on the value channel often beats plain equalization — local contrast without the noise blow-up.

How CLAHE works



tiles, each equalized
then blended

1. Split the image into a grid of **tiles**.
2. Build each tile's histogram; **clip** bars above `clipLimit`, redistribute the excess.
3. Equalize each tile; **bilinearly interpolate** between tile maps to avoid seams.

CLAHE in code

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
out = clahe.apply(gray)              # local contrast, noise held down
```

- `clipLimit`: higher = stronger local contrast (and more noise). 2–3 is a safe start.
- `tileGridSize`: more tiles = more local, but risk of blotchiness on smooth areas.

Note

CLAHE is deterministic given its parameters — good for a repeatable pipeline.

Why equalization can hurt

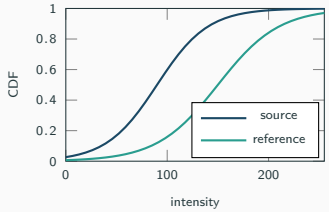
- It's data-driven and **unpredictable**: two similar images can map very differently.
- It amplifies whatever is there — including sensor noise and JPEG blocks.
- For a pipeline, prefer a **deterministic** op (fixed gamma/stretch) so later stages are stable.

Key idea

"Looks better to the eye" $\neq$ "better for the next algorithm." Enhance for the goal, not the demo.

Histogram matching

- Sometimes you want an image's histogram to *match* a reference (consistent look across a dataset).
- Useful when meter photos come from many cameras/lightings and you want uniform input.
- Map via the two CDFs — the same machinery as equalization, but to a target shape.



Math

For each r: find z with $\mathrm{CDF}_{\text{ref}}(z) = \mathrm{CDF}_{\text{src}}(r)$, then map $r \mapsto z$.

Color

Color: enhance value, not each channel

- Equalizing R, G, B *separately* shifts colors (ugly casts).
- Convert to HSV/Lab, enhance the **value/lightness** channel, convert back.
- Keeps hue intact while fixing brightness/contrast — the correct way to enhance color photos.

✕ Pitfall

Per-channel contrast on RGB is a classic beginner mistake that tints the whole image.

The value-channel pipeline




```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(hsv)
v = cv2.createCLAHE(2.0, (8, 8)).apply(v)      # enhance value only
out = cv2.cvtColor(cv2.merge([h, s, v]), cv2.COLOR_HSV2BGR)
```

In-class exercise

In-class exercise (10 min)

Threshold a shadowed meter

You have a meter photo with a diagonal shadow: the top-left is dark, the bottom-right is bright.

1. You run global Otsu and the top-left digits disappear into black. Why?
2. You switch to adaptive with `blockSize=11` and get speckle everywhere. What do you change?
3. Order these for this image: gamma, adaptive threshold, CLAHE. What runs first, and why?

Debrief

- (1) One global T can't fit both lighting zones; the dark side falls entirely below T.
- (2) `blockSize` too small tracks noise — raise it (e.g. 31) and/or raise `C`.
- (3) Enhance **first** (CLAHE/gamma to even out and lift contrast), **then** adaptive threshold.

Key idea

Uneven light $\rightarrow$ local methods. Enhance before you binarize, never after.

Putting it together

The DocScan enhancement→mask chain



- Below T vs above T is a **classification** — errors here propagate to every later step.
- Too high T: digits break up. Too low: background bleeds in. Tune on *hard* images.
- Each stage is deterministic and inspectable — save the intermediate images and look.

Thresholding is a decision, not a default

- Choosing T commits every pixel to one of two classes — there is no going back downstream.
- Global (Otsu) when lighting is even and the histogram is bimodal; adaptive when it isn't.
- Measure, don't guess: overlay the mask on the gray image and check the digit edges.

✓ Good practice

Keep the enhancement deterministic and log the chosen T — reproducibility beats a one-off pretty result.

Wrap-up

What you should now be able to do

- Threshold with a global (Otsu) or adaptive method, and know when each fails.
- Explain Otsu via between-class variance and compute it on a small histogram.
- Use CLAHE and histogram matching, and say why plain equalization can hurt a pipeline.
- Enhance color correctly (value channel, not per-RGB).

This week's deliverable

Due before Week 3 (see `project/`)

On real meter/plate images from `E:/data`:

1. Plot the histogram; classify each image's exposure/contrast (from Session 1).
2. Apply and compare: stretch, equalization, gamma, and CLAHE — with before/after.
3. Produce a clean Otsu *and* adaptive binary of the digits; say which wins and why.
4. Note: for your DocScan image, which enhancement + threshold combo reads best?

Enhance for the goal, then commit with a threshold.

A clean binary mask is what turns a meter photo into readable digits.